Python 3.12 AsyncIO & High-Performance Coroutines

Presented By Prof. Samruddhi

Agenda

- • Introduction to AsyncIO
- • Understanding Coroutines
- • Performance Optimization
- • Advanced Coroutines Patterns

AsyncIO Basics

- • AsyncIO is for asynchronous I/O operations
- • Example: asyncio.sleep()

```
// PYTHON 3.12.8
# Concept: AsyncIO Basics
message = "AsyncIO Basics Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Coroutines

- • Coroutines are lightweight threads
- • Example: async def coroutine()

```
// PYTHON 3.12.8
# Concept: Coroutines
message = "Coroutines Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Example: Simple AsyncIO

- • Example: async def main()

```
            // PYTHON 3.12.8
async def main():
    print('Main function')
    await asyncio.sleep(1)
    print('Main function continues')
result = main()
result
```

Handling Coroutines

- • Using asyncio.gather() to run multiple coroutines
- • Example: asyncio.gather(coroutine1, coroutine2)

```
// PYTHON 3.12.8
# Concept: Handling Coroutines
message = "Handling Coroutines Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Performance Optimization

- • Use asyncio.gather() for better performance
- • Example: asyncio.gather(coroutine1(), coroutine2())

```
// PYTHON 3.12.8
# Concept: Performance Optimization
message = "Performance Optimization Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Advanced Coroutines Patterns

- • Using async/await for cleaner syntax
- • Example: async def main(): await coroutine()

```
// PYTHON 3.12.8
# Concept: Advanced Coroutines Patterns
message = "Advanced Coroutines Patterns Executed in Pytho...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Exercise: Simple AsyncIO Application

- • Create an async function to print numbers from
- • Example: async def print_numbers()

```
            // PYTHON 3.12.8
async def print_numbers():
    for i in range(1, 6):
        print(i)
result = print_numbers()
result
```

Summary

- • AsyncIO for I/O-bound tasks
- • Coroutines for concurrency
- • Performance optimization techniques
- • Advanced patterns for better code

Closing

- • Thank you for attending
- • Questions are welcome
- • References provided